DAT32-4

APIs, Automation & Data Pipelines

Albert School — 16 hours

Contents

Preface

In **Programming in Python** (DAT12-2) you wrote scripts that read a file, processed it, and wrote a result — and you met your first API: an HTTP `GET` with the `requests` library, and a JSON response parsed into a dictionary. In **SQL & Data Querying** (DAT22-5) you learned to model data into tables, to `INSERT`, `UPDATE` and `DELETE`, to reason about keys and duplicates, and to drive a SQLite database from Python with `sqlite3`. In **Software Engineering & Code Design** (DAT22-3) you learned the terminal, virtual environments, project structure, the README as a contract, classes with clean public methods, and `try/except` for failures you expect.

This course joins those three threads into one artefact: a **data pipeline**. A pipeline pulls data from an API you do not control, reshapes it, and stores it somewhere durable — and then does so again tomorrow, and the day after, without a human watching. That last clause is the whole difficulty. A script you run once, by hand, watching the output scroll past, is forgiving: if it crashes you see the traceback and re-run it. A pipeline that runs at 02:00 while you sleep is not. The API may be down; its response format may have changed overnight; your script may have already loaded half the data before it died. The discipline this course installs is **operational**: assume the run will fail in one of a few predictable ways, and build so that the failure is survivable and the re-run is safe.

A note on method. Every idea arrives as a concrete example first and the general statement second, and we keep one running case throughout so each new tool attaches to a system you can already picture.

The running case — StationSync. A city runs a public bike-share scheme. Its open REST API exposes, among other things, a `/stations` endpoint: one record per docking station, with `id`, `name`, `lat`, `lon`, `capacity`, `bikes_available`, and `updated_at`. We are building **StationSync**: a Python pipeline that, every night, fetches every station, cleans the records, and loads them into a SQLite table `stations` so analysts can query availability trends in the morning. Over the next chapters we will authenticate to the API, page through all stations, separate the three pipeline stages, schedule the run, log it, make it survive the three failures that will eventually happen, guarantee that re-running it never duplicates a station, and finally write the README that lets a colleague operate it without reading our code.

1 REST APIs and the request–response model

Before StationSync can do anything, it must **ask the city's server for data**. It does so the same way your browser asks a website for a page: it sends an HTTP request to a URL and waits

for a response. The only difference is that the response is data (JSON) meant for a program, not HTML meant for a human.

Worked example — one request to StationSync's source. StationSync sends:

```
GET https://api.citybikes.example/v1/stations?page=1
```

The server answers with a **status line** (`200 OK`), some **headers**, and a **body**:

```
{ "stations": [
    { "id": 42, "name": "Gare Centrale", "capacity": 30, "bikes_available": 7 },
    ... ],
  "next": "/v1/stations?page=2" }
```

One request, one response. Everything the pipeline does is built out of this single exchange repeated.

Definition 1 A **REST API** exposes resources at URLs called **endpoints**. A client sends an HTTP **request** naming a **method** (`GET` to read, `POST` to create, `PUT`/`PATCH` to update, `DELETE` to remove) and an endpoint; the server returns a **response** carrying a **status code** and, usually, a **body** (here, JSON). The exchange is the **request–response model**: the client always speaks first, and each request is independent — the server remembers nothing between them (it is **stateless**).

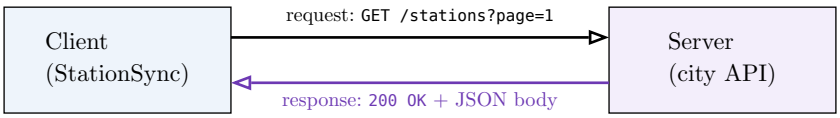

Figure 1: The request–response model. The client always initiates; the server replies once and forgets. Every pipeline action is this exchange, repeated.

Every response carries an **HTTP status code**: a three-digit number whose **first digit** tells you the category of outcome. Reading it correctly is the first branch in every robust pipeline — it decides whether you parse the body, fix your request, or retry later.

Definition 2 HTTP status codes by leading digit:

- **2xx — success.** The request worked. `200 OK` (body returned), `201 Created`, `204 No Content`.
- **3xx — redirection.** The resource lives elsewhere; the client should follow. `301 Moved Permanently`, `304 Not Modified`.
- **4xx — client error. Your** request was wrong; retrying it unchanged will fail again. `400 Bad Request`, `401 Unauthorized`, `403 Forbidden`, `404 Not Found`, `429 Too Many Requests`.
- **5xx — server error.** The **server** failed; your request may be fine and worth retrying later. `500 Internal Server Error`, `502 Bad Gateway`, `503 Service Unavailable`.

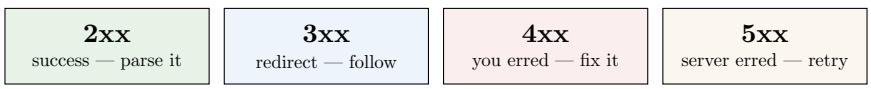

Figure 2: The four status-code families. The 4xx/5xx split is the one that governs pipeline behaviour: a 4xx means **don't bother retrying**, a 5xx means **retrying later might work**.

Common pitfall The `requests` library does **not** raise an exception on a 4xx or 5xx response — from its point of view the HTTP exchange succeeded, it just carried a failure code. `r = requests.get(url)` followed by `r.json()` on a `404` page will give you garbage or a parse error, not a clear "not found". Always branch on `r.status_code` (or call `r.raise_for_status()`) **before** trusting the body.

2 API authentication

The city's API is public, but it still wants to know **who** is calling — to enforce rate limits and to revoke abusers. So before StationSync gets any data it must **prove its identity**. There are two mechanisms you will meet, and the difference between them is a difference in how much damage a leaked secret can do.

Worked example — the same request, authenticated two ways. With an API key — one fixed secret string, sent on every request as a header:

```
headers = {"Authorization": f"Bearer {API_KEY}"}
r = requests.get(url, headers=headers)
```

With OAuth — first exchange your credentials for a short-lived **access token**, then send that:

```
tok = requests.post(TOKEN_URL, data={
    "grant_type": "client_credentials",
    "client_id": CLIENT_ID, "client_secret": CLIENT_SECRET,
}).json()["access_token"]
r = requests.get(url, headers={"Authorization": f"Bearer {tok}"})
```

The request that fetches stations looks identical; what differs is the lifetime and blast radius of the secret it carries.

Definition 3

- An **API key** is a single, long-lived secret string that identifies the caller. It is sent with each request (as a header or query parameter) and stays valid until it is manually revoked.
- An **OAuth token** (an **access token**) is a short-lived credential obtained by exchanging your client credentials through an authorization flow. It is sent with each request, **expires** after minutes or hours, is **scoped** to specific permissions, and is renewed by **refreshing** rather than by re-sharing the underlying secret.

The security difference follows directly from those lifetimes. An API key is a **static, long-lived** secret: if it leaks — committed to Git, pasted in a ticket, logged by accident — whoever finds it has full access until a human notices and revokes it, which may be never. An OAuth access token is **short-lived and scoped**: a leaked token expires on its own in minutes, and it grants only the narrow permissions it was issued for. OAuth also lets a service act on a user's behalf **without ever holding the user's password**. The trade-off is complexity: OAuth requires a token-exchange step and logic to refresh an expired token mid-run.

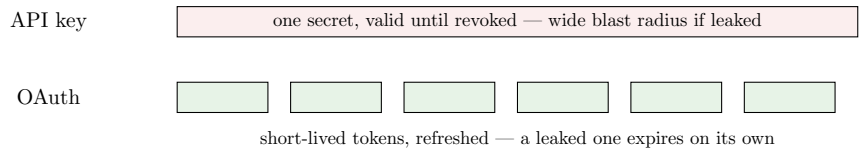

Figure 3: An API key is one secret that lives until revoked; OAuth issues a stream of short-lived, scoped tokens. The narrower each token's lifetime, the smaller the damage a leak can do.

Common pitfall Whichever mechanism you use, the secret must **never** be hard-coded or committed to version control. Read it from an **environment variable** (or a `.env` file that is in `.gitignore`): `API_KEY = os.environ["CITYBIKE_KEY"]`. A key pushed to a public repository is compromised within minutes — bots scan GitHub for exactly this — and rotating it afterwards is far more painful than keeping it out in the first place.

3 Pagination

StationSync's city has 1,800 stations, but the `/stations` endpoint returns only 50 per call. This is universal: an endpoint that could return thousands of records never returns them all at once — it returns one **page** and tells you how to ask for the next. Collecting the whole dataset means **looping until the API says there is no more**.

Worked example — walking every page of stations.

```
stations = []
url = "https://api.citybikes.example/v1/stations?page=1"
while url is not None:
    r = requests.get(url, headers=headers)
    r.raise_for_status()
    body = r.json()
    stations.extend(body["stations"])   # accumulate this page
    url = body.get("next")               # None on the last page → loop ends
```

The loop does not stop after a fixed number of pages; it stops when the API's own `next` pointer is absent. After it finishes, `stations` holds all 1,800 records.

Definition 4 **Pagination** is the division of a large result set into successive **pages**. The client requests them in turn — by following a server-provided `next` link (**cursor / link** style), by incrementing a `page=` number (**offset** style), or by passing back a **cursor** token — and **accumulates** the records until the API signals that no page remains. The assembled result is then stored in a structured form (a list of records, a CSV, or a database table, as in DAT12-2 and DAT22-5).

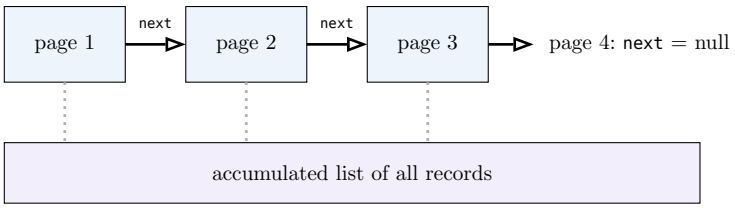

Figure 4: Pagination is a loop, not a single call. Each page points to the next; the records pile into one accumulator; the loop ends when `next` is absent.

Common pitfall The single most common pagination bug is **stopping after the first page** — you test against an endpoint that happens to fit on one page, ship it, and silently lose 97% of production data. The loop's exit condition must be the API's own end-of-results signal (`next` is null, the page comes back empty, or the returned count is below the page size), **never** a hard-coded page count. A close cousin: forgetting to handle `429 Too Many Requests` mid-loop — many APIs rate-limit paging, and you must pause and retry rather than treat the 429 as the end.

4 ETL pipeline design

We now have the pieces to fetch all stations. The temptation is to write one long function that fetches, cleans, and inserts in a single pass. Resist it. A pipeline is built as **three clearly separated stages**, because each stage fails for different reasons, is tested differently, and changes for different reasons.

Worked example — StationSync as three functions.

```
def extract():                      # talk to the API; return raw records
    ...                             # (pagination lives here, nothing else)
    return raw_stations

def transform(raw_stations):        # clean & reshape; no network, no database
    rows = []
    for s in raw_stations:
        rows.append({
            "id": int(s["id"]),
            "name": s["name"].strip(),
            "capacity": int(s["capacity"]),
            "bikes_available": int(s["bikes_available"]),
        })
    return rows

def load(rows, conn):               # write to the database; no cleaning
    ...

def run():                          # the only place the three meet
    load(transform(extract()), conn)
```

Notice what each stage is **forbidden** to do: `extract` never cleans, `transform` never touches the network or the database, `load` never reshapes. The data flows one way: extract → transform → load.

Definition 5 **ETL** names the three stages of a pipeline:

- **Extract** — acquire raw data from the source (here, the paginated API).
- **Transform** — clean, reshape, validate, and deduplicate the data in memory.
- **Load** — write the transformed data into the destination (here, SQLite).

Each stage is a separate, independently testable unit, with data flowing strictly one way through them.

Figure 5: The ETL pipeline. Three boxes, one direction. Keeping them separate is what lets you re-run just the failing stage and unit-test each in isolation, using the class and function discipline from DAT12-2 and DAT22-3.

This separation is not bureaucracy; it pays off the moment something breaks. If the API changes a field name, only `transform` is wrong. If the database is locked, only `load` failed and you can re-run it on the already-extracted data. Mixing the stages forfeits all of that: a single 200-line function that fails at line 150 gives you no clean place to resume.

5 Scheduling

StationSync must run **every night**, whether or not anyone remembers to launch it. Running it by hand defeats the purpose. The operating system can run a script on a timetable for you: **cron** on Linux and macOS, **Task Scheduler** on Windows.

> **Worked example — a crontab line that runs StationSync nightly at 02:00.**
>
> ```
> 0 2 * * *  /home/data/stationsync/.venv/bin/python /home/data/stationsync/run.py
> >> /home/data/stationsync/logs/cron.log 2>&1
> ```
>
> Read the five fields left to right: minute `0`, hour `2`, **every** day of month, **every** month, **every** day of week. So: "at 02:00, every day." The rest is the command — note the **absolute** path to the virtual-environment's Python, and the redirection that appends all output to a log file.

Definition 6 **Scheduling** is the automatic execution of a program at defined times, performed by the operating system rather than a person. A **cron** schedule is a **crontab** entry: five time fields — **minute, hour, day-of-month, month, day-of-week** — followed by the command to run. A field is a specific value, a `*` (every), a list (`1,15`), a range (`1-5`), or a step (`*/10`). **Task Scheduler** expresses the same idea on Windows through a trigger-and-action configuration.

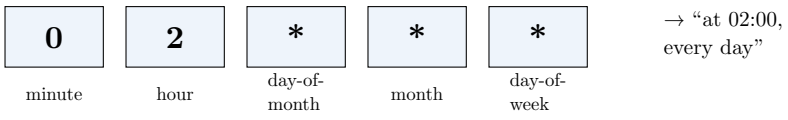

Figure 6: Anatomy of a cron schedule. Five time fields, then the command. * means "every"; here only the minute and hour are pinned, so it fires once a day at 02:00.

Common pitfall Cron runs your script in a **bare environment**: a minimal `PATH`, a different working directory, and none of your shell's variables. Three failures follow, and all are invisible when you test by hand. (1) `python run.py` finds the wrong (or no) interpreter — use the **absolute path to the venv's Python**. (2) Relative paths like `open("data.csv")` resolve against the wrong directory — use absolute paths or `cd` first. (3) Secrets you set in your shell are absent — load them explicitly. "Works when I run it, fails under cron" is almost always one of these three.

6 Logging

When StationSync runs at 02:00 unattended and you find an empty table at 09:00, you have one question — **what happened?** — and the only thing that can answer it is what the script recorded **while it ran**. You cannot re-run it to "see the error", because the API's state has changed since; the night's evidence is gone unless the script wrote it down. That written record is a **log**.

> **Worked example — StationSync narrating its own run.**
>
> ```
> import logging
> logging.basicConfig(
>     filename="logs/stationsync.log", level=logging.INFO,
>     format="%(asctime)s %(levelname)s %(message)s")
>
> logging.info("run started")
> logging.info("extracted %d stations across %d pages", len(raw), pages)
> ...
> logging.error("schema check failed: missing field 'capacity'")
> ```
>
> A morning reader sees, without touching the code:
>
> ```
> 2026-06-19 02:00:01 INFO  run started
> 2026-06-19 02:00:14 INFO  extracted 1800 stations across 36 pages
> 2026-06-19 02:00:15 ERROR schema check failed: missing field 'capacity'
> ```
>
> The cause, the time, and the stage are all there. No re-run needed.

Definition 7 **Logging** is the recording of timestamped, severity-tagged messages describing a program's execution — which stage it reached, what it processed, and what went wrong — to a file or stream that can be inspected after the fact. Python's `logging` module attaches a **timestamp** and a **level** (`DEBUG`, `INFO`, `WARNING`, `ERROR`, `CRITICAL`) to each message and can route it to a file.

The goal is a precise standard: **a failure must be diagnosable from the log alone, without re-running the code**. That standard tells you what to log — the start and end of each stage, the counts that should add up (pages fetched, records extracted, rows loaded), and every caught exception with enough context to locate it.

Common pitfall Do not use `print` for a scheduled pipeline. `print` has no timestamp, no severity, and vanishes unless something happens to capture stdout — and under cron, by default, nothing does. Worse, `print` cannot be turned down: `logging` lets you keep `DEBUG` detail in the file while showing only `WARNING`-and-above on screen, by changing one `level=` argument rather than deleting statements.

7 Pipeline failure modes

A pipeline reading from a source you do not control **will** fail — not as a rare accident but as a routine event you design for. Three failures dominate, and the OP requires StationSync to handle all three. The point is not to prevent them (you cannot) but to make each one **survivable**: detected, logged, and either recovered or failed cleanly without corrupting the database.

> **Worked example — the three failures StationSync will eventually hit.**

- **One night the API is down.** `GET /stations` returns `503` (or the connection times out). Without handling, the script crashes mid-run; with handling, it logs the outage, retries a few times, and exits cleanly leaving last night's data intact.
- **One morning a field is renamed.** The city ships `bikesAvailable` instead of `bikes_available`. The transform's `s["bikes_available"]` raises `KeyError` — or worse, a default silently writes wrong data.
- **Every re-run duplicates.** You re-run after a partial failure and every station is inserted a second time, double-counting the city.

Definition 8 The three most common **pipeline failure modes**:

- **Source unavailability** — the API does not respond, times out, or returns a 5xx status. **Handle by** detecting the failure (status code / exception), retrying with backoff a bounded number of times, then failing cleanly with a logged message rather than crashing or writing partial data.
- **Schema changes** — the response structure changes: a field is renamed, removed, retyped, or nested differently, breaking the transform. **Handle by** validating the response shape **before** transforming and logging a precise error naming the missing or unexpected field.
- **Duplicate records** — the same records are loaded more than once (e.g. on a re-run or an overlap). **Handle by** making the load **idempotent** (next chapter).

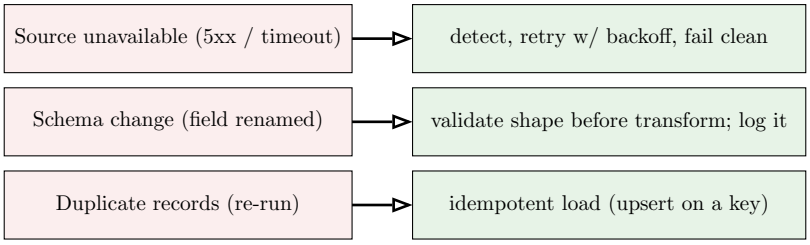

Figure 7: The three required failure modes, each with its defence. None is prevented — each is detected and contained so a single bad night never corrupts the table or stops tomorrow's run.

Common pitfall The dangerous response to a failure is the **silent** one. A bare `except: pass` around the extract turns a source outage into an empty result that then **wipes the table on load** — a far worse outcome than crashing. Likewise a `.get("capacity", 0)` that papers over a renamed field writes plausible-looking zeros that no one notices for weeks. Catch **specific** exceptions, log them, and let the pipeline stop rather than proceed on bad data.

8 Idempotency

The third failure mode — duplicates — deserves its own chapter, because the property that defends against it is the one that makes a pipeline **safe to re-run at all**. After any failure, your instinct is to run StationSync again. That instinct is only safe if a second run on the same data changes nothing.

Worked example — the same run, twice. StationSync loads 1,800 stations. The load uses a blind insert:

```
cur.execute("INSERT INTO stations VALUES (?,?,?,?)", row)   # NOT idempotent
```

Re-run it and the table now holds **3,600** rows — every station twice. Analysts' counts double. Now make the load **idempotent** using `id` as the primary key:

```
cur.execute("""INSERT INTO stations (id, name, capacity, bikes_available)
               VALUES (:id, :name, :capacity, :bikes_available)
               ON CONFLICT(id) DO UPDATE SET
                 name=excluded.name, capacity=excluded.capacity,
                 bikes_available=excluded.bikes_available""", row)
```

Re-run it now and the table still holds exactly **1,800** rows — existing stations are updated in place, none duplicated, none lost. The second run is a no-op on the row count.

Definition 9 A pipeline is **idempotent** when running it more than once on the same input produces the **same end state** as running it once — no duplicated records and no lost records — regardless of how many times, or how far through, previous runs got. It is the property that makes re-running after a failure **safe**.

Idempotency is achieved by giving each record a stable **unique key** (here the station `id`) and loading with an **upsert** — **insert if the key is absent, update if it is present** — rather than a blind `INSERT`. This is exactly the key-and- deduplication reasoning from SQL (DAT22-5), now doing operational work: `ON CONFLICT … DO UPDATE` (SQLite), or a delete-then-insert inside a transaction, or a staging-table merge all express the same idea.

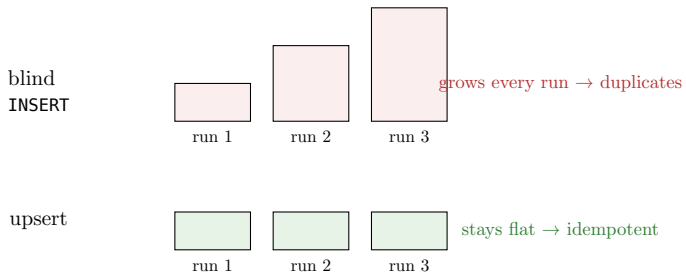

Figure 8: Row count across three identical re-runs. A blind `INSERT` grows the table each time; an upsert on the record's key leaves it unchanged after the first run — the definition of idempotency.

Common pitfall Idempotency is a property of the **whole** pipeline, not just the SQL verb. An upsert keyed on a column that is not actually unique still duplicates; an auto-incrementing surrogate id that ignores the record's natural key duplicates; appending to a CSV is never idempotent. Always key on something **intrinsic and stable** about the record (the station's own `id`), and **verify the property the only honest way — run the pipeline twice and check the row count is identical.**

9 Pipeline documentation

StationSync now works, survives failure, and is safe to re-run. The last deliverable is the one that lets **someone other than you** operate it. When the pipeline breaks at 02:00 on a night you are on holiday, a colleague must be able to understand, run, and fix it **without reading the code**. That is what the documentation is for — it is the pipeline's contract, in the README spirit of DAT22-3.

Worked example — StationSync's README, in miniature.

- **Inputs** — Source: city bike API, `GET /v1/stations` (paginated). Auth: API key in env var `CITYBIKE_KEY`. Runs nightly at 02:00 via cron.
- **Outputs** — SQLite database `stations.db`, table `stations(id PK, name, capacity, bikes_available)`. One row per station; `id` is the unique key.
- **Failure behaviour** — On API outage: retries 3×, then exits without touching the table; see `logs/stationsync.log`. On schema change: aborts before loading and logs the offending field. Re-running is **safe** — the load is idempotent (upsert on `id`), so a second run never duplicates or drops rows.
- **To run by hand** — `CITYBIKE_KEY=… .venv/bin/python run.py`.

A colleague can operate, schedule, and troubleshoot from these four headings without opening `run.py`.

Definition 10 **Pipeline documentation** states, in plain language a colleague can act on:

- the **inputs** — the source, endpoints, required credentials, and schedule;
- the **outputs** — the destination, the resulting schema, and the unique key;
- the **failure behaviour** — what happens on source unavailability, on a schema change, and on a re-run, and where to look (the logs);
- **how to run and recover it** by hand.

It must be readable, and the pipeline operable, **without reading the code**.

Common pitfall The test of pipeline documentation is brutal and specific: **hand it to a colleague who has never seen the code and ask them to recover from last night's failure.** If they must open `run.py` to learn where the logs are, what the unique key is, or whether a re-run is safe, the documentation has failed — no matter how elegant the code behind it.

StationSync, end to end, one line per stage: send a request and read its status code → authenticate with a secret kept out of source control → page until the API says stop → split the work into extract / transform / load → schedule it with cron → log every stage so failures are diagnosable without a re-run → detect and contain the three failure modes → make the load idempotent so re-running is always safe → document inputs, outputs, and failure behaviour so a colleague can operate it blind.

Exercises

1. Explain what a REST API is and describe the request–response model. Given a list of HTTP status codes (e.g. `200`, `301`, `404`, `429`, `503`), state for each whether it indicates success, redirection, a client error, or a server error, and say which ones a pipeline should retry.

2. Authenticate against an API once using an API key and once using an OAuth token. Explain the security difference between the two, and show how you keep the secret out of source control.

3. Paginate through a paginated API endpoint, accumulate all results, and store them in a structured format. Verify you collected every record, not just the first page, and explain how your loop's exit condition guarantees that.

4. Design a Python pipeline with extraction, transformation, and loading as three clearly separated functions (or classes). For each stage, state one thing it is forbidden to do, and explain how the separation helps when one stage fails.
5. Schedule a Python script to run automatically every day at a fixed time using cron (Linux/ macOS) or Task Scheduler (Windows). Write the crontab line, explain each field, and name two reasons a script that works by hand might fail under cron.
6. Add logging to the pipeline so that a failure can be fully diagnosed from the log file without re-running the code. Log the start/end of each stage and the counts that should reconcile. Explain why `print` is inadequate here.
7. Make the pipeline handle the three common failure modes: source API unavailability (detect, retry, fail clean), a schema change in the response (validate before transforming), and duplicate records. Show what your code does in each case and why it never proceeds on bad data.
8. Make the pipeline idempotent so that re-running it on the same data produces no duplicates and loses no records. Implement the load as an upsert on a stable unique key, and **verify** idempotency by running the pipeline twice and checking the row count is identical.
9. Write documentation for the pipeline stating its inputs, outputs, failure behaviour, and how to run and recover it, such that a colleague can operate and troubleshoot it without reading the code. Test it by having someone follow it unaided.